Horizon Sweeps

A single call of `solve()` uses one horizon pair $H = [H_1, H_2]$ (Section **??**). A *horizon sweep* solves one scenario for several pairs and compares the solved cases. Two functions implement the sweeps: `sweep_operating_horizons` varies H_2 for one H_1, and `sweep_horizon_grid` varies H_1 and H_2 on a grid. No input key starts a sweep, and `solve()` never runs one. Both functions are exported by the package (Section 3.1).

Cost Measures and the Ranking Criteria

Let x, z, x^r be the solution of a case. The cost of day k is

$$c_k = C_M \sum_{i=1}^{F} x_{i0k} + \sum_{i=1}^{F} \sum_{\ell=1}^{L} \big(C_{R,i\ell}\, z_{i\ell k} + C_{\mathrm{rep},i\ell}\, x^r_{i\ell k} \big), \tag{1}$$

where $z_{i\ell k} = 0$ for a cell with $C_{R,i\ell} = 0$ (no variable z is built, Section **??**). The cost of the transitory phase and the cost of the operating phase are

$$J_{\mathrm{init}} = \sum_{k=1}^{H_1} c_k, \qquad J_{\mathrm{op}} = \sum_{k=H_1+1}^{T} c_k, \qquad \bar{J}_{\mathrm{op}} = J_{\mathrm{op}} / H_2. \tag{2}$$

The peak damage is $u = \max_{i,k} \sum_\ell \mu_{i\ell k}$, and the damage penalty is $C_D\, u$. The result keys are `J_initialization`, `J_op`, `J_op_average` ($= \bar{J}_{\mathrm{op}}$), `peak_damage` and `damage_penalty` (Section **??**).

A solve with `objective_mode = "operating_average"` minimises $\bar{J}_{\mathrm{op}} + C_D\, u$ (Section **??**). The transitory phase has no cost in this objective, but it is still constrained. With `objective_mode = "evaluation_total"` and an *evaluation horizon* $E > H_1$ (days), the solve minimises $P_E + C_D\, u$, where P_E is the *projected cost*

$$P_E = J_{\mathrm{init}} + (E - H_1)\, \bar{J}_{\mathrm{op}}. \tag{3}$$

P_E is the cost of the transitory phase, then of the operating phase repeated until day E. Two result keys carry it: `projected_evaluation_cost` holds P_E, and `evaluation_objective` holds $P_E + C_D\, u$.

A case is *feasible* when its result holds a solution and its status is not `"infeasible"`, not `"unbounded"` and not `"inf_or_unbounded"`. The *MIP gap* is the relative gap between the objective of the best solution found and the best bound (MIP: mixed-integer program). A case is *proven* when it is feasible and Gurobi reports the status `"optimal"` at the MIP gap used for that case, that is, the run option `mip_gap` of Section **??**. A proven case is optimal within that gap, not exactly optimal. The sweep report records the requested gap of each case, so a reader sees how strong the proof is.

Ranking criteria. `sweep_operating_horizons` ranks cases by $\bar{J}_{\mathrm{op}}$. The loop constraint (Section **??**) makes the operating phase repeat, so $\bar{J}_{\mathrm{op}}$ is the cost per day of the repeating regime. It is comparable across values of H_2, whereas a raw objective over T days is not. `sweep_horizon_grid` ranks by $\bar{J}_{\mathrm{op}}$ when no evaluation horizon is given. With an evaluation horizon it ranks by P_E, which also prices the transitory phase, so it is the criterion that compares different values of H_1. In both functions the *best proven case* is the proven case with the smallest ranking value, and the *best feasible case* is the feasible case with the smallest ranking value. A tie goes to the case evaluated first.

Remark (what the proof certifies). The status `"optimal"` certifies the objective that the solve minimised. This objective is $\bar{J}_{\mathrm{op}} + C_D\, u$ in the mode `operating_average` and $P_E + C_D\, u$ in the mode `evaluation_total`, while the ranking value is $\bar{J}_{\mathrm{op}}$ or P_E alone. The sweep therefore ranks by cost and uses the damage penalty only inside each solve.

`sweep_operating_horizons`

```
sweep_operating_horizons(
    input_path, operating_horizons, *, output_path=None,
    stop_on_gradient=False, gradient_tolerance=1e-3,
    flat_gradients_required=2, minimum_cases=3,
    maximum_mip_gap_for_stopping=None,
    warm_start=False, warm_start_mode="start", n_workers=1,
    objective_mode="operating_average", evaluation_horizon=None) -> dict
```

Argument	Meaning
`input_path`	Input file (YAML, JSON or HDF5). H_1 is the first entry of the input key `H`, or its value when `H` is a scalar. The second entry of `H` is ignored.
`operating_horizons`	Candidate values of H_2: positive integers, at least one, no duplicates. The function evaluates them in increasing order.
`output_path`	Report file, with the suffix `.yaml` or `.yml`. The function creates missing parent folders. `None`: no file is written.
`stop_on_gradient`, `gradient_tolerance`, `flat_gradients_required`, `minimum_cases`, `maximum_mip_gap_for_stopping`	Early stop (Section 1.3). Required: `gradient_tolerance` ≥ 0 and finite, `flat_gradients_required` ≥ 1, `minimum_cases` ≥ 2, and `maximum_mip_gap_for_stopping` is `None` or a finite number ≥ 0.
`warm_start`, `warm_start_mode`	Warm start between cases (Section 2).
`n_workers`	Number of cases solved at the same time (Section 1.5).
`objective_mode`, `evaluation_horizon`	Written into every case input, replacing the values of the input file. The mode `evaluation_total` needs $E > H_1$.

Behaviour.

1. The function reads the input file with the reader of `solve()` (Section 3.1). It accepts the same formats and raises the same file errors. The data must be a mapping (`TypeError` otherwise).
2. The function checks every argument of the table above before it solves any case. An invalid argument raises `ValueError`. An argument error is never recorded as a case error. It also raises `ValueError` when `H` is a list of a length other than two, or when `warm_start` is true and `n_workers` > 1, or when `stop_on_gradient` is true and `n_workers` > 1.
3. For each candidate H_2, the case input is the input data with `H` $= [H_1, H_2]$, `objective_mode` and (if given) `evaluation_horizon` replaced. Nothing else changes: the function does not interpolate time-indexed data. A profile whose length differs from H_2 (and does not broadcast) is rejected by the input checks of Section **??**. This rejection is an error of that case only.
4. The case is solved as `solve()` solves an input file, with `results_path=None`. The sweep keeps the summary of the case (Section 1.4), not its schedule.
5. After each case the function appends the case row, annotates the gradient (Section 1.3), tests the stopping rule, and writes a checkpoint when `output_path` is given (Section 1.4).
6. At the end the function writes the final report and returns it.

Early Stop

Let $\bar{J}_n$ and $H_{2,n}$ be the cost per day and the operating horizon of the n-th evaluated case. When cases $n-1$ and n are both feasible, the *relative gradient* of case n is

$$\hat{g}_n = \frac{\bar{J}_n - \bar{J}_{n-1}}{(H_{2,n} - H_{2,n-1})\,\max\{\,|\bar{J}_{n-1}|,\ \varepsilon_{\text{mach}}\}}, \tag{4}$$

with $\varepsilon_{\text{mach}}$ the machine precision of double-precision numbers. Case n is classified `"increase"` if $\hat{g}_n >$ `gradient_tolerance`, `"decrease"` if $\hat{g}_n <$ `−gradient_tolerance`, and `"flat"` otherwise. If one of the two cases is not feasible, or if $n = 1$, the class is `"unavailable"`. A case *qualifies* when it reports a finite MIP gap that is at most its gap threshold. The threshold of a case is `maximum_mip_gap_for_stopping` if that argument is a number, and otherwise the MIP gap used for that case (the entry `requested_mip_gap` of its row). With the default, a case qualifies when Gurobi has met the gap that was requested for it. The gradient of case n is *eligible to stop* when both cases $n-1$ and n qualify. The function annotates every case with these values, also when `stop_on_gradient` is false.

With `stop_on_gradient = True` the function stops after case n when $n \geq$ `minimum_cases`, the gradient of case n is eligible to stop, and one of these holds:

- the class of case n is `"increase"` (stopping reason `"cost_increase"`);
- the class of case n is `"flat"`, and the last `flat_gradients_required` cases up to n all have the class `"flat"` and an eligible gradient (stopping reason `"flat_gradient"`).

The last candidate in `operating_horizons` is always a hard limit. A case that does not qualify, for example one that stopped at the time limit above its requested gap, is recorded, but it cannot trigger a stop.

Report, Checkpoint and Errors

The report is a mapping of plain Python types. The file is written with a safe YAML writer, so it contains no arrays and no Python objects.

Key	Content
`input`	Path of the input file, as given.
`complete`	`false` in a checkpoint, `true` in the final report (also after an early stop).
`last_completed_H2`	Largest H_2 among the evaluated cases; `null` before the first case.
`objective`, `varied_parameter`	Text that names the minimised objective, and the string `"H2"`.
`fixed_dimensions`, `H1`	F, M, L, H_1.
`planned_operating_horizons`, `evaluated_operating_horizons`	The sorted candidates, and the H_2 of the cases in `cases`.
`warm_start`, `n_workers`	`enabled`, `mode`; number of workers.
`stopping_rule`	`enabled`, `relative_gradient_tolerance_per_H2`, `flat_gradients_required`, `minimum_cases`, `maximum_mip_gap_for_stopping` (`null` when the default applies), `reason` (`"in_progress"`, `"candidate_range_exhausted"`, `"cost_increase"` or `"flat_gradient"`), `stopped_early` (the run is complete and fewer cases than planned were evaluated).
`cases`	One row per evaluated case, ordered by H_2 (rows below).

`formulation_growth`	Between the first and the last case that carry formulation counts: `from_H2`, `to_H2`, `added_time_steps`, and for each count key q of the row entry `formulation`, the entries `added_`q and q`_per_added_time_step`. `null` when fewer than two cases carry counts or both have the same T.
`best_proven_H2`, `best_proven_J_op_average`	Best proven case (`null` if none).
`best_feasible_H2`, `best_feasible_J_op_average`, `best_feasible_status`	Best feasible case (`null` if none).

Entry of a case row	Content
`H1`, `H2`, `T`, `dimensions`	Horizons and the sizes F, M, L, H_1, H_2, T.
`status`, `objective_mode`, `evaluation_horizon`	As in the result of the case.
`objective`, `J_op`, `J_op_average`, `J_initialization`, `peak_damage`, `damage_penalty`, `operating_objective`, `evaluation_objective`, `J_total`, `projected_evaluation_cost`	Result keys of the case (Section **??**).
`objective_bound`, `mip_gap`, `absolute_mip_gap`	Best bound; relative gap; the difference $\max\{0,$ `objective` $-$ `objective_bound`$\}$.
`requested_mip_gap`, `time_limit_seconds`, `optimizer_seconds`, `solutions_found`	The run options used and the optimizer outcome.
`warm_start`	The warm-start report of the case (Section 2).
`formulation`	Model size from the `performance` entry of the result: `variables`, `continuous_variables`, `integer_variables`, `binary_variables`, `linear_constraints`, `general_constraints`, `indicator_constraints`, `quadratic_constraints`, `nonzeros`.
`timing`	`model_construction_seconds`, `optimizer_call_seconds`, `solution_extraction_seconds`, `backend_wall_seconds`, `branch_and_bound_nodes`, `simplex_iterations`, `work_units`.
`cost_gradient_per_H2`, `relative_cost_gradient_per_H2`, `gradient_classification`, `gradient_stopping_eligible`	Equation (4) and its class (Section 1.3).

Checkpoint. When `output_path` is given, the function rewrites the report after every case. Until the sweep ends, the report has `complete = false` and the stopping reason `"in_progress"`. If a job ends before the run is complete, the file is a valid report of the cases evaluated so far. In parallel mode the function recomputes the gradient annotations over the rows ordered by H_2 at each rewrite.

Error capture. The function catches every exception (class `Exception`) that a case raises after the arguments are accepted, for example a failed input check of the case or a solver error such as a missing licence. It logs the error at level ERROR with H_1 and H_2, records a row with the status `"error: "` followed by the message and `null` in every other entry except the sizes, and continues with the next case. Such a row is neither feasible nor proven. An error does not stop the sweep, and a warm start never uses the result of a failed case (Section 2).

Parallel Cases

With `n_workers` > 1 the function solves up to `n_workers` cases at the same time, each in its own process with its own Gurobi environment. Each running solve uses one Gurobi licence token, so a licence with one token allows only `n_workers` $= 1$. The cases do not use each other's results, so the option is refused together with warm starts between cases, and together with the early stop, which needs the preceding cases before it starts the following ones. Rows arrive in the order in which the cases finish; the report orders them by H_2.

`sweep_horizon_grid`

```
sweep_horizon_grid(
    input_path, transitory_horizons, operating_horizons, *,
    output_path=None, warm_start=True, evaluation_horizon=None,
    **sweep_options) -> dict
```

The function evaluates every pair (H_1, H_2) of the two lists. Both lists contain positive integers; the function sorts them and removes repeated values. If `evaluation_horizon` is given, it must exceed every candidate H_1 (`ValueError` otherwise). The keyword options in `sweep_options` are those of Section 1.2 except `objective_mode`, which the grid sets itself. Because `warm_start` is true by default, a call with `n_workers` > 1 must also pass `warm_start=False`.

For each H_1 in increasing order, the function runs `sweep_operating_horizons` on the input data with that H_1 and the list of H_2. It sets `objective_mode` to `"evaluation_total"` when an evaluation horizon is given, and to `"operating_average"` otherwise. The first case of each H_1 starts without warm start; the following cases use the nearest preceding case of the same H_1 that has a solution. The early stop applies within each H_1 and uses $\bar{J}_{\mathrm{op}}$ as in Section 1.3. Case errors are captured as in Section 1.4. The function rewrites the grid report after every case of every H_1 (with `complete = false`) and at the end.

Key of the grid report	Content
`input`, `complete`	As in Section 1.4.
`objective`, `comparison_metric`	Text that names the minimised objective; the ranking key, `"projected_evaluation_cost"` (P_E) or `"J_op_average"`.
`mip_certificate_metric`	The string `"objective"`: the proof rule refers to the solved objective (remark in Section 1.1).
`evaluation_horizon`	E, or `null`.
`varied_parameters`, `fixed_dimensions`	`["H1", "H2"]`; F, M, L.
`planned_transitory_horizons`, `planned_operating_horizons`	The sorted candidates.
`warm_start_enabled`	The value of `warm_start`.
`H1_sweeps`	The report of each operating sweep, one per H_1. Its key `input` names the input file of the grid.
`cases`	All case rows, ordered by H_1 and then H_2.
`best_proven`, `best_feasible`	`null`, or a mapping with `H1`, `H2`, `T`, `J_initialization`, `J_op_average`, `peak_damage`, `damage_penalty`, `objective`, `operating_objective`, `evaluation_objective`, `projected_evaluation_cost`, `status`, `mip_gap` of the selected case.

Warm Starts

A *warm start* gives Gurobi values for the binary variables, so that the search begins from a known schedule. A warm start changes the time of a solve, not the model: it does not change

the feasible set or the objective. The package has four mechanisms:

1. a *start* from the schedule of a previous result (Gurobi attribute `Start`), with an optional completion;
2. *hints* from the schedule of a previous result (Gurobi attribute `VarHintVal`);
3. a *relaxation start*, which needs no previous result;
4. a *greedy heuristic*, which constructs a schedule from the input and returns it in the form of item 1.

Interface

`solve()` (Section 3.1) has two keyword arguments for warm starts: `warm_start` (default `None`) and `warm_start_mode` (default `"start"`). The argument `warm_start` is a *source*: a dictionary with the keys of a result, or the dictionary that the greedy heuristic returns (Section 2.7). A source must carry the arrays `x` (shape $F' \times (M'+1) \times T'$) and the integers `H1`, `H2` with $T' = H_1' + H_2'$; it may carry `m` and `r` (shape $F' \times L' \times T'$). A source without a solution (`x` is `null`) is accepted and has no effect. A result with a solution found before a time limit is a valid source. The sizes F', M', L' may differ from the problem's sizes: the values are set for the indices that both have.

`warm_start_mode` is `"start"`, `"hint"` or `"both"`; another value raises `ValueError`. The relaxation start is selected by the run options `relaxation_warm_start` and `relaxation_time_limit` (Section **??**).

Day Mapping

The source has the horizons H_1', H_2' and the problem has H_1, H_2. Day $k \in \{1, \dots, T\}$ of the problem takes the values of the source day

$$\sigma(k) = \begin{cases} k, & k \le \min\{H_1, H_1'\}, \\ H_1' + 1 + \big(\max\{0,\ k - H_1 - 1\} \bmod H_2'\big), & \text{otherwise.} \end{cases} \tag{5}$$

So the transitory days keep their index, and the operating days repeat the operating phase of the source cyclically. A shorter operating phase can seed a longer one. When $H_1 > H_1'$, the days $H_1'+1, \dots, H_1$ all take the first operating day of the source. When the source has the same sizes F, M, L, the mapped schedule copies whole days of the source, so it keeps the one-activity rows, the mission coverage and the depot-day rows of every day. It need not satisfy the dynamics or the loop rows of the horizons of the problem. For each day k, train i and cell (i, ℓ) the mapped values are $x_{ijk} \leftarrow x'_{ij\sigma(k)}$ for $j = 0, \dots, M$, $x^m_{i\ell k} \leftarrow m'_{i\ell\sigma(k)}$ (value 0 if the source has no `m`), and $x^r_{i\ell k} \leftarrow r'_{i\ell\sigma(k)}$ when replacement is allowed (`allow_replacement`, Section **??**) and the source has `r`. Every value is rounded to the nearest integer. Each other binary variable that these values fix through its defining rows (the selector $n_{i\ell k}$ of the indicator form, Eq. (**??**)) receives that value.

Start, Hints and Completion

Mode `"start"`. The solve sets the attribute `Start` of every mapped variable (Section 2.2). The continuous variables get no value. Gurobi tries to complete the start by itself, with limited effort, and discards a start that it cannot make feasible.

Completion. If the source has the key `complete_binary_start` with the value `true`, the solve completes the start before it optimises. The time limit of the completion is the source key `completion_time_limit` (seconds, default 60). The steps are:

1. Copy the model. In the copy, fix every binary variable that received a `Start` value to that value.
2. Optimise the copy. All binary values are fixed, so the copy has only continuous variables.

3. If the copy has a solution, set the attribute `Start` of every variable of the original model to the value in the copy. The start is then a complete, feasible point of the original model.
4. Otherwise record the status of the copy. If the copy is infeasible, compute an *irreducible infeasible subset* (IIS) of the copy: a set of rows that is infeasible, and that becomes feasible when any one row is removed (Gurobi function `computeIIS`). Record the names of its linear rows and general constraints, at most 50 names.

The solve never relaxes or changes the original model, and it never stops because a completion failed: the optimisation starts with the binary values only. A completion is requested by the source, not by an argument of `solve()`. The greedy heuristic requests it.

Mode `"hint"`. The solve sets the attribute `VarHintVal` of every mapped variable to its mapped value, and sets no `Start` and no completion. Gurobi uses hints to guide its heuristics and its branching during the whole solve. A hint needs no feasible point, so it also helps when the mapped schedule does not fit the horizons of the problem and Gurobi discards it as a start. The hint priority `VarHintPri` is not set.

Mode `"both"`. The solve sets `Start` and `VarHintVal` on the same variables, and performs the completion if the source requests it.

Relaxation Start

If `relaxation_warm_start` is true and no start or hint was applied (the entry `applied` of `warm_start` is false), the solve computes a start without a source:

1. Copy the model, and relax its linear rows with Gurobi's function `feasRelaxS`, which minimises the sum of the absolute violations. The relaxation keeps integrality, variable bounds and general constraints.
2. Optimise the relaxed copy with the time limit $\max\{1,$ `relaxation_time_limit`$\}$ seconds and the MIP gap 0.05.
3. If the copy has a solution, set `Start` of every integer variable of the original model to the rounded value of the variable of the same name in the copy.

The relaxed solution is not a feasible point of the original model. It gives integer values only, and the original model must find its own first solution.

Warm-Start Reports

The result of a solve carries two dictionaries (Section **??**).

Key	Content
`warm_start.applied`	`true` when at least one value was set.
`warm_start.values`	Number of values set.
`warm_start.mode`	The mode used, or `null` without a source.
`warm_start.source_horizon`	`H1` and `H2` of the source, or `null`.
`warm_start.completion`	`requested`; `status` (`"not_requested"`, `"completed"`, `"infeasible"`, `"infeasible_or_unbounded"`, `"time_limit"` or `"status_`q`"` with the Gurobi status code q); `fixed_integer_variables`; `values_copied`; `runtime_seconds`; `iis_constraints` (list of names, empty unless the status is `"infeasible"`).

`relaxation_warm_start`	`enabled`; `status` (`"disabled"` when the option is false, `"skipped"` when it is true and a start was applied, otherwise the Gurobi status of the relaxed copy as a string); `relaxation_objective`; `values_copied`; `runtime_seconds`.

Warm Starts in Sweeps

The sweep functions (Section 1) with `warm_start=True` use, for each case, the result of the nearest preceding case of the same H_1 that has a solution as the source of the solve, with the mode `warm_start_mode`. A case that raised an error or has no solution is skipped. The first case of a sequence has no source. Warm starts between cases need the preceding results, so they are refused together with parallel cases (Section 1.5).

Greedy Heuristic

```
build_greedy_warm_start(cfg, *, repair_trigger=0.70)
    -> (dict, GreedyWarmStartReport)
```

The function is in the module `fleet_management.greedy_warm_start`; the package does not export it. The argument `cfg` is the normalised configuration of an input (Section **??**). The trigger `repair_trigger` must lie in $[0, 1]$ (`ValueError` otherwise). The function works for every model of this specification. It builds a schedule that tries to satisfy the rows of the model. It does not try to give a low cost. It returns a source for `solve()`: the dictionary with the keys `H1`, `H2`, `x`, `m`, `r`, `complete_binary_start` (`true`) and `completion_time_limit` (60), and a report.

Construction pass. Keep a proxy state $\hat{s}_{i\ell}$ for each cell, with the initial value $\mu_{0,i\ell}$. Treat the days $k = 1, \ldots, T$ in order.

1. Order the missions by decreasing difficulty $\max_{i,\ell} \mu^{\text{inc}}_{i\ell jk}/\tau_{i\ell}$. A tie goes to the mission with the larger index.
2. Take the missions in this order. Give each one to the train i that has no mission yet on day k and minimises the pair $\big(\max_\ell(\hat{s}_{i\ell} + \mu^{\text{inc}}_{i\ell jk})/\tau_{i\ell},\ \sum_\ell(\hat{s}_{i\ell} + \mu^{\text{inc}}_{i\ell jk})/\tau_{i\ell}\big)$ in lexicographic order. A tie goes to the smaller index i. Increase $\hat{s}_{i\ell}$ by $\mu^{\text{inc}}_{i\ell jk}$ for the train and day. The input check $F > M$ (Section **??**) guarantees a train for every mission.
3. Let r be the value of `repair_trigger`. A train without a mission stays idle, except when some component has $\hat{s}_{i\ell} > r\,\tau_{i\ell}$. Then the train goes to the depot ($x_{i0k} = 1$), every such component is repaired ($x^m_{i\ell k} = 1$), and $\hat{s}_{i\ell} \leftarrow (1 - \rho_{i\ell})\,\hat{s}_{i\ell}$ for it.

The pass schedules no replacement. The proxy uses the whole-state repair map for every cell, whatever its repair law.

Loop pass. After the construction pass the missions are fixed, so the cells are independent except through the depot days of a train. For each cell, compute its reference trajectory ζ (Section **??**) for the current schedule, with the starting values, the starting anchors and the replacement values of the inputs. The *excess* e of the cell is the largest of

- $\max\{0,\ \zeta^q_T - \zeta^q_{H_1}\}/M_q$ for every tracked quantity and anchor q with natural range M_q (the loop row, Section **??**);
- the violation of the reliability row of the cell in its exact form, on every day, divided by the natural range of the quantity that the row bounds (Sections **??**–**??**).

While $e > 10^{-10}$, the pass tries every *free day* $k \in \{H_1 + 1, \ldots, T\}$, a day on which the train has no mission and the cell has no intervention. It puts a repair of the cell on the free day (the

train goes to the depot on that day) and recomputes the excess. It keeps the repair that gives the smallest excess; a tie goes to the later day. If no repair lowers e by more than 10^{-10} and replacement is allowed, the same trial is made with a replacement. If no trial lowers e, the pass counts one failure for the cell and goes to the next cell. Repairs and replacements of a cell never use a day on which the train flies a mission.

Report. `GreedyWarmStartReport` is a record; its method `as_dict()` returns the fields below as plain Python types.

Field	Content
`repairs`, `replacements`	Number of ones in `m` and in `r`.
`repeatability_feasible`	`true` when the loop pass counted no failure.
`repeatability_failures`	Number of cells with a failure.
`maximum_repeatability_excess`	Largest raw excess (in the units of its quantity) over all cells after the loop pass.
`maximum_proxy_ratio`	Largest value of $\hat{s}_{i\ell}/\tau_{i\ell}$ in the construction pass and of the reference mean over $\tau_{i\ell}$ in the loop pass.
`assignment_order`	For each day, the list of missions (indices $1, \ldots, M$) in the order that the pass treated them.

A start with failures is still returned. Its completion can then end with the status `"infeasible"` with an IIS.

File & Folder Structure

The package `fleet_management` has one module per task. The shared Gurobi model and the registry of cell builders are in `base.py`; each degradation model has its own module.

```
Fleet_Management/
    pyproject.toml                Python >= 3.10; extra "dashboard" (
       streamlit)
    README.md
    spec/spec.tex, references.bib this specification
    examples/                     example inputs and regression scripts
    tests/                        pytest suite
    euler/                        two Slurm job scripts of the ETH Euler
        cluster
    src/fleet_management/
        __init__.py               public API (Section "Public API")
        config.py                 input reading, checks and broadcasting
           : load_config -> FleetConfig
        solver.py                 solve(): file checks, dispatch,
           serialisation (YAML, JSON, HDF5)
        horizon_sweep.py          sweep_operating_horizons,
           sweep_horizon_grid
        greedy_warm_start.py      build_greedy_warm_start
        validator_dashboard.py    Streamlit entry point of the
           dashboards
        degradation_model/
            base.py               shared model, registry, objective,
               warm starts, result extraction,
                                  performance counters
            gaussian.py           cell builder "gaussian"
            inverse_gaussian.py   cell builder "inverse_gaussian"
            wiener.py             cell builder "wiener"
            gamma.py              cell builder "gamma"; computation of
               the cap A_1
```

```
            rainflow.py            cell builder "rainflow" (five bounds)
            gamma_utils/
                gamma_replay_validator.py       replay validator
                gamma_stochastic_validator.py   Monte Carlo validator
                gamma_tail_validator.py         exact validator
                gamma_tail_bound.py
                   moschopoulos_tail_probability
        utils/
            plotter.py             plot_management, plot_horizon_sweep
        validation/
            validator.py           validate
            dashboard_inspector.py, dashboard_gamma.py
```

Cluster jobs. The folder `euler/` holds two Slurm job scripts (Slurm is the job scheduler of the Euler cluster of ETH Zürich). Each runs one script of `examples/regression/`:

- `gamma_horizon_convergence.sbatch` runs `run_horizon_sweep.py`;
- `vbz_progressive_feasibility.sbatch` runs `run_progressive_vbz_feasibility.py`.

Both scripts call only the public API of Section 3.1. The behaviour that this specification defines does not depend on them.

Public API

The package exports the functions of the table.

Function	Defined in
`solve`	this section
`plot_management`	Section 3.3
`plot_horizon_sweep`	Section 3.3
`sweep_operating_horizons`	Section 1
`sweep_horizon_grid`	Section 1
`validate`	Section 4
`validate_gamma_replay_schedule`	Section 4.3
`validate_gamma_replay_files`	Section 4.3
`validate_gamma_stochastic_schedule`	Section 4.3
`validate_gamma_schedule`	Section 4.3
`validate_gamma_schedule_files`	Section 4.3

```
solve(input_path, results_path="output.yaml", *,
      warm_start=None, warm_start_mode="start") -> dict
```

`solve()` reads one input file, builds one Gurobi model for the whole fleet, solves it, and returns the result dictionary of Section **??**. Arrays in the dictionary are NumPy arrays; Gurobi objects are not stored in it.

1. The input file must exist (`FileNotFoundError`) and have the extension `.yaml`, `.yml`, `.json`, `.h5` or `.hdf5` (`ValueError`).
2. If `results_path` is not `None`, the solve resolves it before it reads the input. A path without an extension gets the extension `.yaml`. The extension selects the format of the result file: YAML, JSON or HDF5 (as for inputs); another extension raises `ValueError`. The folder of the result file must exist (`FileNotFoundError`) and be writable (`PermissionError`). With `results_path=None` no file is written.
3. The *input reader* reads the file by its extension into a mapping of the keys of Section **??**. The horizon sweeps and `validate` use the same reader.

4. `load_config` normalises the mapping and runs the checks of Section **??**. A failed check raises `ValueError`, and no model is built.
5. The solve builds the model (Section 3.2), applies the warm start (Section 2), optimises, extracts the solution, writes the result file if requested, and returns the dictionary.

Every fleet, whatever its models, goes through the same route: no model has a separate solver. Gurobi errors, such as a missing licence, are not caught.

Cell-Builder Interface

The shared model of `base.py` is built once for the whole fleet. Each cell (i, ℓ) is then filled in by the *cell builder* of its model (the entry of `model` in the input). Builders are registered in a dictionary by model name.

```
register_cell_builder(name, builder)    # CELL_BUILDERS[name] = builder
get_cell_builder(name)                  # NotImplementedError if name is
    not registered
dispatch_cell(ctx, i, l)                # get_cell_builder(ctx.model_of[i
    , l]).add_cell(ctx, i, l)
```

The names are `gaussian`, `inverse_gaussian`, `wiener`, `gamma` and `rainflow`. The error of `get_cell_builder` lists the registered names. A builder is an object with the attribute `name` and the hooks below.

Hook	Duty
`prepare(ctx, cfg, cells, opts)`	Called once for each model that occurs in the fleet, with the list `cells` of the cells of that model. It stores the per-cell arrays of the model in the context and creates the variables of the model's tracked quantities, anchors and selectors, for these cells only (no variable that no row uses). It may tighten variable bounds, and it sets Gurobi's parameter `NonConvex` to 2 when some cell of the model keeps an exact quadratic row.
`add_cell(ctx, i, l)`	Adds every row of one cell that Section **??** assigns to it: the dynamics in the form selected by `encoding` (Section **??**), the replacement and anchor rows, the repair-cost rows (Section **??**), the reliability rows in the form selected by `reliability_impl` (Section **??**), and the loop rows of the full tracked state (Section **??**).
`extract(ctx, cfg, out)`	Optional. Adds model-specific entries to the result dictionary `out`, after the shared entries are extracted.

The *context* `ctx` carries the sizes (F, L, M, H_1, H_2, T), the Gurobi model, the shared variables (x of shape $F \times (M+1) \times T$; x^m and x^r of shape $F \times L \times T$, with x^r absent when replacement is not allowed; the mean μ of shape $F \times L \times T$; z at the cells with $C_R > 0$; u when $C_D > 0$), the per-cell arrays (model, τ, ε, ρ, initial and replacement values, starting anchors), the increment accessor, the resolved run options and a dictionary `extras` for storage of a model. The increment accessor returns, for train i, mission j (index from 0 for the depot), cell ℓ and day k, the increment after the phase lookup of Section **??**.

Building order.

1. Create the model, apply the run options, and create the shared variables.
2. Call `prepare` once for each model present, in the alphabetical order of the names.
3. Build the objective (Section **??**) and the rows that do not depend on the model: the assignment rows (Section **??**), the depot-day rows (Section **??**) and the damage-regularisation rows (Section **??**).
4. Call `add_cell` for every cell, with i in the outer loop and ℓ in the inner loop.

Every builder must define the shared variable $\mu_{i\ell k}$ for all $k = 1, \ldots, T$ of each of its cells, because the damage regularisation and the output read it. After the optimisation, the shared extraction reads the variables, and then each model's `extract` hook runs. A model is added by writing a module with its builder and registering it, by adding its input checks to `config.py` (Section **??**), and by adding its section to this specification.

Plotting API and Layout

The package has one schedule plot and one sweep plot, both in `utils/plotter.py`.

```
plot_management(input_file_path, plot_file_path=None) -> None
```

Parameter	Description
`input_file_path`	Result file of `solve()`, in YAML, JSON or HDF5 format. A missing file raises `FileNotFoundError`; another extension raises `ValueError`.
`plot_file_path`	Image file, `.png` or `.pdf`. Default `"output.png"`. A path without an extension gets `.png`. Another extension raises `ValueError`; a missing folder, `FileNotFoundError`; a folder without write permission, `PermissionError`.

The function plots any result that holds a solution, whatever its status: an incumbent found before a time limit is plotted, and the status is shown in the summary panel. It raises `ValueError` when the result lacks one of the keys `F`, `M`, `L`, `H1`, `H2`, `tau`, `mu_0`, `mu`, `x` (names the missing keys), when a shape differs from Section **??**, or when $H_1 + H_2 \neq T$. The keys `m` and `r` are read when present and taken as all zero otherwise. The names of the components come from `component_names` (default "Component ℓ" with ℓ from 1); a list of a length other than L raises `ValueError`. The model of each cell comes from the key `model` (Section **??**).

Plot Layout

The figure has a grid on the left and a summary panel on the right.

- **Rows.** One row for each cell (i, ℓ), ordered by train and then by component: $F\,L$ rows. The label of a row has two lines: "Train i — *component name*" and the model name of the cell (`gaussian`, `inverse_gaussian`, `wiener`, `gamma` or `rainflow`). A heavier horizontal line separates the trains.
- **Columns.** $T + 1$ columns: "initial" (the values $\mu_{0,i\ell}$) and the days $1, \ldots, T$. A vertical dashed line after day H_1 separates the phases. Above the grid, the labels "transitory" and "operating" name the two parts.
- **Colour.** The background of a field is a green-to-red scale of $\mu_{i\ell k}/\tau_{i\ell}$, clipped to $[0, 1]$ (green 0, red 1). A colour bar is labelled "tracked mean / failure threshold". The values are the tracked means, which are upper bounds of the expected degradation (Section **??**).
- **Marks.** Each field of a day carries one mark. The first rule that applies decides:

Mark	Rule for cell (i, ℓ) on day k
`P`	replacement: $x^r_{i\ell k} = 1$
`R`	repair: $x^m_{i\ell k} = 1$
`M`$_j$	train i flies mission j: $x_{ijk} = 1$ with $j \geq 1$ (the mark shows j)
`D`	depot day without intervention: $x_{i0k} = 1$, $x^m_{i\ell k} = 0$, $x^r_{i\ell k} = 0$
`I`	idle: the train has no activity on day k (all $x_{ijk} = 0$)

A legend below the grid names the five marks, with a distinct colour for each. A value is 1 when it is above 0.5. The column "initial" has no mark.

Summary panel (title "Run summary"). Each line is a label and a value; a line is omitted when its value is missing from the result. Status (coloured by outcome); Objective; Dimensions F, M, L; Horizon H_1, H_2, T; Models (the distinct model names); Repair (the distinct values of `repair_model`); Objective mode and J_{op}/H_2, if `objective_mode` is in the result; MIP gap in percent; the numbers of variables, continuous variables, integer variables and linear rows; optimizer time; branch-and-bound nodes; repaired components; replaced components. The model counts are those of Gurobi's presolved model when the result carries them (`presolved_performance`), and those of the original model otherwise; the label says which. When the result carries no presolved counts and a file `runner.out` (a Gurobi log) lies in the folder of the result file, the plot reads the counts from the lines "Presolved: r rows, c columns" and "Variable types: a continuous, b integer". The numbers of repaired and replaced components are the numbers of ones in `m` and `r` (components, not depot visits).

```
plot_horizon_sweep(sweep_report_path, plot_file_path=None) -> None
```

`sweep_report_path` is one path, or a list of paths, of reports of `sweep_operating_horizons`. A report is a YAML or JSON file. Another file type raises `ValueError`, a missing file raises `FileNotFoundError`, and an empty list raises `ValueError`. The default image file is `"horizon_sweep.png"`; the rules for `plot_file_path` are those of `plot_management`. Reports in a list must agree in F, M, L and H_1 (`ValueError` otherwise), and each needs a list `cases`. When several cases have the same H_2, the plot keeps the one with a solution, then the smaller J_{op}/H_2, then the larger objective bound. A report without a case raises `ValueError`. The figure has three panels over H_2 (the cases are joined by lines that are not extrapolations):

(a) J_{op}/H_2 for each case, with a different mark for a proven case (status `"optimal"`), a feasible case that is not proven, and a case without a feasible solution (including an error row); the best feasible case is marked and the values of the proven cases are written;
(b) the relative MIP gap in percent. When the entry `maximum_mip_gap_for_stopping` of `stopping_rule` is a number, a horizontal line shows it; otherwise a mark shows the requested MIP gap of each case;
(c) the numbers of continuous variables, integer variables and linear constraints.

The title names the sweep and the sizes F, M, L, H_1.

Validation Tools

The validation tools check a saved result against its input. They do not call the optimiser and they read no Gurobi object. There are three groups: the structural validator for every model (Section 4.1), the dashboards (Section 4.2), and the validators of the Gamma model (Section 4.3). All tools read the input with the input reader of `solve()` and normalise it with `load_config` (Section 3.1), and read a result file in the formats that `solve()` writes. They read the model of each cell from the result key `model`.

Structural Validator

```
validate(input_path, results_path, validation_path="validation.yaml",
         tol=1e-6) -> dict
```

`validate` checks that the saved schedule and states satisfy the rows of the formulation that do not need the solver, and writes and returns a report. The report file has the extension `.yaml`, `.yml` or `.json` (a path without an extension gets `.yaml`; another extension raises `ValueError`). The function creates the folder of the report; it raises `PermissionError` if the folder is not writable. It raises `FileNotFoundError` for a missing input or result file, and `ValueError` for the following: an unsupported extension, a result without a solution (`x` is `null`), an input that fails

the checks of Section **??**, and a result whose sizes F, M, L, H_1, H_2 or whose array shapes differ from those of the input (the message names the quantity). The solver status is not a check: a feasible incumbent found before a time limit is validated like an optimal solution, and the report echoes the status.

A check passes when its *violation* is at most `tol`. The checks are:

Name	Statement	Violation
`binary`	x, x^m, x^r take only the values 0 and 1	largest distance of an entry from $\{0, 1\}$
`one_activity`	$\sum_{j=0}^{M} x_{ijk} \le 1$ for all i, k (Section **??**)	$\max\{0,\ \max_{i,k}(\sum_j x_{ijk} - 1)\}$
`mission_coverage`	$\sum_i x_{ijk} = 1$ for $j = 1, \dots, M$ and all k; the depot $j = 0$ is not summed	$\max_{j \ge 1,k} \lvert \sum_i x_{ijk} - 1 \rvert$
`depot_gating`	$x^m_{i\ell k} + x^r_{i\ell k} \le x_{i0k}$ (Section **??**); $x^r = 0$ if `allow_replacement` is false	largest excess of the left over the right side
`idle`	if the result has `idle`: $\texttt{idle}_{i\ell k} = x_{i0k} - x^m_{i\ell k} - x^r_{i\ell k}$	$\max \lvert \texttt{idle} - (x_{i0k} - x^m - x^r) \rvert$
`damage_rows`	if $C_D > 0$: the rows of Section **??** hold for the saved u and μ	largest violation of a row
`loop_`q	the loop row (Section **??**) $q_{i\ell T} \le q_{i\ell H_1}$ for each tracked quantity or anchor $q \in \{\texttt{mu}, \texttt{v}, \texttt{R}, \texttt{K}, \texttt{g_mu}, \texttt{g_v}, \texttt{g_R}, \texttt{g_K}\}$. The result carries every one of them, with `NaN` (in a file `null`) at the cells that do not track it; these entries are skipped	$\max\{0,\ \max_{i,\ell}(q_{i\ell T} - q_{i\ell H_1})\}$
`objective`	the objective recomputed from the saved arrays equals the saved `objective`	relative difference of the two values, with the divisor $\max\{1, \lvert \texttt{objective} \rvert\}$

The recomputed objective uses the day cost c_k of Eq. (1) with the saved x, z, x^r and the costs of the input, and the saved u. For `objective_mode` equal to `total` it is $\sum_{k=1}^{T} c_k + C_D\, u$; for `operating_average` it is $J_{\text{op}}/H_2 + C_D\, u$; for `evaluation_total` it is the projected cost P_E of Eq. (3) (Section **??**). The check does not test the optimality of the schedule. It tests that the reported objective is the cost of the reported solution.

The loop checks use the saved tracked states, which satisfy the loop row, and not the reference trajectory: the reference trajectory of a feasible schedule can violate the loop row (Section **??**).

The report is a mapping with the keys `status`, `solver_objective`, `recomputed_objective`, `tolerance`, `checks` (a list of mappings with `name`, `passed`, `violation`, `tolerance`), `passed` (all checks passed) and `max_violation` (the largest violation).

Dashboards

The dashboards are Streamlit pages (Streamlit is a Python library for web dashboards; it is in the extra `dashboard`). The entry point is `validator_dashboard.py`; the command is `python -m streamlit run src/fleet_management/validator_dashboard.py`. A sidebar selects the mode, the input file, the result file and the tolerance (default 10^{-6}), and a button loads the pair. The dashboards read only the input and the result: they need no log file. A failed load shows the error on the page; no page is drawn from partial data.

Mode "Result inspector" (every model). Tabs:

1. *Overview*: status, objective, the models present, H_1, H_2, T, the number of reliability violations, the largest μ/τ, and the cells with the smallest margin.

2. *Reliability*: a table of the margin of every (train, component, day), with filters for train, component and violations only.
3. *Repeatability*: for each cell and each saved tracked state, the value of the loop row (Section **??**).
4. *Train timeline*: the mean of each component of one train over the days, with the threshold τ and the marks of Section 3.3.
5. *Heatmap*: the margins on the grid of cells and days.
6. *Component comparison*: one component across the trains.
7. *Validation*: the checks of `validate` (Section 4.1).
8. *Raw data*: the table behind the tabs, with an export.

The *margin* of a cell on a day is the right side minus the left side of the reliability row of its model and bound, in its exact form, with the τ, ε and bound of that cell and the saved tracked states (Sections **??**–**??**). A margin of at least $-$`tolerance` means that the row holds. The exact form is used also when the solve used the tangent or piecewise-linear form, so the margin shows the true slack. The result carries every tracked quantity; the dashboard leaves out the entries that are `NaN`. The margins of cells with different models and bounds are not comparable in size; only the sign is.

Mode "Gamma diagnostic" (Gamma cells). Tabs: an overview with the counts of repairs and replacements and the outcome of the replay validator; the tracked mean of each Gamma cell with its cap $A_1(\beta^*\tau, \varepsilon)/\beta^*$ (Section **??**); the exact tail probability of the true damage against ε (Section 4.3); a heatmap of this tail over the cells and days; and the raw data.

Validators of the Gamma Model

The three validators of this section check the Gamma cells of a result. Each takes the normalised configuration `cfg` (Section **??**) and the result dictionary, or an input path and a result path, and returns a report (a mapping of plain Python types). Each raises `ValueError` when `cfg` has no Gamma cell, when a required array is missing, has a wrong shape or has a non-finite entry, and when an array that must be binary is not within `tolerance` of $\{0, 1\}$. The argument `tolerance` (default 10^{-8}) must be positive. With `raise_on_failure=True` a report with `valid = false` raises `AssertionError`. The file versions take also `report_path` (default `None`): the report is written there as YAML or JSON (`ValueError` for another suffix), and the folder is created if it is missing. The tracked state of a Gamma cell is the mean μ and, under ARD_1, its anchor g^μ (Section **??**).

Replay validator.

```
validate_gamma_replay_schedule(cfg, result, *, tolerance=1e-8,
                               raise_on_failure=False) -> dict
validate_gamma_replay_files(input_path, result_path, report_path=None,
                            **kwargs) -> dict
```

It replays the bookkeeping of the saved schedule. It reads the keys `x`, `m`, `r` (optional), `mu`, `z`, and the anchor `g_mu` of the mean for the cells that use ARD_1. Global violations (strings in `global_violations`): a result horizon different from T; a train with more than one activity; a mission not covered exactly once; repair and replacement of a cell on one day; an intervention on a day with a mission or without a depot day. For each Gamma cell and day k the validator computes the reference trajectory of the mean and of its anchor (Section **??**) from the saved schedule, with the initial values, the starting anchor and the replacement value of the cell. The *event* of the day is `"replacement"`, `"repair"`, `"mission"` or `"idle"`. The errors, by field, are:

Field	Error

`tracked_mean`	$\max\{0,$ reference mean $-$ `mu`$_{i\ell k}\}$: a tracked mean below the reference breaks the upper-bound property of the one-sided rows (Section **??**). A tracked mean above the reference is slack and is not an error.
`mean_anchor`	the same for the anchor `g_mu`, under ARD_1
`removed_mean`	$\lvert z_{i\ell k} - x^m_{i\ell k}\,\rho_{i\ell}(\mu_{i\ell,k-1} - g_{i\ell,k-1})\rvert$ computed from the saved states, with $g = 0$ under ARD_∞ (Section **??**); $z = 0$ on a day without repair
`cap`	$\max\{0,$ `mu`$_{i\ell k} - A_1(\beta^*\tau, \varepsilon)/\beta^*\}$ (Section **??**)
`repeatability`	$\max\{0,$ `mu`$_{i\ell T} -$ `mu`$_{i\ell H_1}\}$, and the same for the saved anchor

An error above `tolerance` adds a mapping `i`, `l`, `k`, `event`, `field`, `expected`, `saved`, `error` to `violations`. The report has the keys `validator`, `valid` (no global violation and no violation), `solver_status`, `dimensions`, `gamma_cells`, `gamma_ard1_cells`, `transitions_checked`, `repairs`, `replacements`, `maximum_errors` (the largest error of each field), `global_violations`, `violations` and `timing`.

Exact validator (module `degradation_model.gamma_utils.gamma_tail_validator`; the package does not export it).

```
validate_gamma_tail_bound_schedule(cfg, result, *, tolerance=1e-8,
    convolution_tolerance=1e-12, max_series_terms=100000,
    include_steps=False, raise_on_failure=False) -> dict
validate_gamma_tail_bound_files(input_path, result_path, report_path=
   None,
                                **kwargs) -> dict
```

It checks the safety claim itself, for the saved schedule and without the tracked states. For each Gamma cell it rebuilds the true damage on each day $k = 1, \ldots, T$ as a list of independent Gamma terms, as in the proof of the class of true states (Section **??**): every mission adds a term with the exact rate of its increment; a repair multiplies the rate of the repairable terms by $1/(1-\rho)$ (the shape is unchanged; for $\rho = 1$ the repairable terms disappear); under ARD_1 the repaired terms join the frozen part of the anchor; a replacement starts a new list with the replacement term. An initial or replacement damage without a rate (the inputs `gamma_beta_0` or `gamma_beta_new` are omitted) is a known constant c and is not a term: the tail is then $\Pr(\text{sum of the terms} > \tau - c)$, which is 1 if $c > \tau$. It computes the tail $\Pr(D_k > \tau)$ of the sum of the terms with the Moschopoulos series. The series writes the law of a sum of independent Gamma variables with different rates as a mixture of Gamma laws with non-negative weights. When all rates are equal, the tail is the Gamma tail. The partial sum of the series is a lower bound of the tail. The validator adds the mixture weight that remains, which is at most `convolution_tolerance`, and so obtains a certified upper bound of the tail. If the series does not reach this tolerance within `max_series_terms` terms, the validator raises `RuntimeError`. The argument `convolution_tolerance` must lie in $(0, 1)$ and `max_series_terms` must be positive. A day passes when the upper bound is at most $\varepsilon_{i\ell}$ + `tolerance` and the saved mean is at least the exact mean $\mathbb{E}D_k$ minus `tolerance`. Under ARD_1 the saved anchor must also be at least the exact mean of the frozen part, minus `tolerance`. (The saved values may be larger: they are upper bounds.) The report adds, to the keys of the replay report, `minimum_reliability_slack` ($\min(\varepsilon -$ tail upper bound)), `worst_reliability` (the step that attains it), `maximum_remaining_mass`, the numbers of series terms, and the timing of the tail computations. With `include_steps=True` the report has the list `steps` with the terms and values of every cell and day.

Monte Carlo validator.

```
validate_gamma_stochastic_schedule(cfg, result, *, repetitions=100000,
    random_seed=20260908, maximum_schedule_failure_rate=None,
    confidence_level=0.95, batch_size=20000, tolerance=1e-8,
    raise_on_failure=False) -> dict
```

```
validate_gamma_schedule(cfg, result, *, mode=None, repetitions=None,
    random_seed=None, maximum_schedule_failure_rate=None,
    confidence_level=None, batch_size=None, tolerance=1e-8,
    raise_on_failure=False) -> dict
validate_gamma_schedule_files(input_path, result_path, report_path=None,
                              **kwargs) -> dict
```

The schedule stays fixed. In each of `repetitions` runs, the validator draws the true damage of every Gamma cell: the initial damage, each mission increment and each replacement draw are independent Gamma variables with the inputs' means and rates (the initial damage under ARD_1 is the sum of two independent terms with the means g_0^{μ} and $\mu_0 - g_0^{\mu}$, the first one being the anchor). An initial or replacement damage without a rate (the inputs `gamma_beta_0` or `gamma_beta_new` are omitted) is the known constant of its mean. Each day it applies the repair law of the cell to the sampled path: $D \leftarrow (1-\rho)\,D$ under ARD_∞; $D \leftarrow g + (1-\rho)(D-g)$ then $g \leftarrow D$ under ARD_1; a replacement draws the damage of the replacement part and, under ARD_1, sets g to it. After each day it records whether $D > \tau$. A *schedule failure* is a run in which some Gamma cell exceeds its threshold on some day. The draws use a generator seeded with `random_seed` and are made in batches of `batch_size` runs; the same arguments give the same report. `confidence_level` must lie in $(0, 1)$, `repetitions` and `batch_size` must be positive, and `maximum_schedule_failure_rate` (if given) must lie in $[0, 1]$.

The report counts the failures of the fleet, of each cell and of the worst day of each cell. Each rate comes with its one-sided Clopper–Pearson upper confidence bound at `confidence_level`. For f failures in n runs the bound is the `confidence_level`-quantile of the Beta$(f+1,\ n-f)$ law, and 1 if $f = n$. The keys are:

Key	Content
`validator`, `mode`, `valid`, `timing`	Name of the validator, the mode, the verdict, the time used.
`scope`, `complete_fleet_validation`	"Gamma cells only"; false if the fleet has cells of other models.
`excluded_cells`	For each cell that is not a Gamma cell: i, l, component, model, reason.
`repetitions`, `random_seed`, `batch_size`, `confidence_level`	The arguments used.
`schedule`	failed_replays, failure_rate, failure_rate_upper_confidence_bound, maximum_failure_rate, observed_target_met, confidence_qualified_target_met.
`cells`	For each Gamma cell: i, l, component, repair_model, threshold, analytical_epsilon, failed_replays, failure_rate and its upper confidence bound, worst_time_step with the failures, the rate and the bound of that day, maximum_sampled_damage, maximum_threshold_excess.

The report is valid when the observed schedule failure rate is at most the target `maximum_schedule_failure_rate`, or when no target is given. The result is an empirical stress test and not a proof: zero failures in n runs does not show that the true rate is below $1/n$.

`validate_gamma_schedule` selects the validators by `mode`: `"deterministic"` (the replay validator; the default), `"stochastic"` (the Monte Carlo validator), or `"both"` (a report with the keys `deterministic` and `stochastic` and `valid` true only if both are valid). Another value raises `ValueError`. An argument that is `None` takes the default of the Monte Carlo validator. The Monte Carlo validator samples Gamma cells only. It lists every other cell in `excluded_cells` with the reason "not a Gamma cell".

References

References

[1] L. Doyen and O. Gaudoin. Classes of imperfect repair models based on reduction of failure intensity or virtual age. *Reliability Engineering & System Safety*, 84(1):45–56, 2004. doi:10.1016/S0951-8320(03)00173-X. Read in the authors' preprint submitted on 25 October 2002, whose page numbers are cited.

[2] S. Mercier and I. T. Castro. Stochastic comparisons of imperfect maintenance models for a gamma deteriorating system. *European Journal of Operational Research*, 273(1):237–248, 2019. doi:10.1016/j.ejor.2018.06.020.

[3] F. Roosta-Khorasani and G. J. Székely. Schur properties of convolutions of gamma random variables. *Metrika*, 78:997–1014, 2015. doi:10.1007/s00184-015-0537-9. Read in arXiv:1601.04731v1, whose page numbers are cited.

[4] G. J. Székely and N. K. Bakirov. Extremal probabilities for Gaussian quadratic forms. *Probability Theory and Related Fields*, 126:184–202, 2003. doi:10.1007/s00440-003-0262-6.